

TECHNICAL DEEP DIVE

混合模型系统构建：开源语义路由器的架构演进与工程实践

从两层网关的语义盲区到信号驱动的多模型协作 (MoM) 抽象

基于技术演讲与配套材料整理

配套材料：vllm-sr-oss.pdf

2026-09-30

原视频：[vLLM 语义路由器：探索多模型系统智能的边界](#) · 配套资料：[vLLM-SR 课件](#)

当企业同时对接数十家模型供应商、自建多套推理集群、还要兼顾边缘设备部署时，一个贯穿全栈的矛盾日益尖锐：每条用户请求都携带丰富的意图信息，但负责调度的网关只能看到 Token 计数和队列深度，对请求“想做什么”毫无感知。本文围绕 vLLM-SR (vLLM Semantic Router，语义路由器) 项目，从这一工程矛盾出发，沿“碎片化诊断 → 语义层定位 → 最小可行验证 → 信号驱动架构 → 多模型协作 → 工业证据 → 虚拟化抽象”的因果链路逐层展开，梳理开源社区在语义路由方向上的架构演进与关键设计决策。

适读人群：AI 基础设施工程师、后端架构师、大模型应用开发者及技术负责人。

前置知识：

- 了解大模型推理基础概念（如 KV Cache、Prefix Cache）
- 熟悉 API 网关与负载均衡的基本原理
- 对当前开源与闭源大模型生态的碎片化现状有初步认知

阅读目标：

- 理解传统两层 AI 网关在处理复杂语义请求时的结构性局限
- 掌握 vLLM-SR 语义路由层的系统定位及其与推理引擎的职责边界
- 解析从静态分类到六阶段信号驱动架构的演进因果
- 理解级联、融合、工作流三种多模型协作机制的工程实现
- 认识混合模型（MoM）虚拟化抽象在端、云、企多环境中的部署价值

一、算力碎片化与传统网关的语义盲区

工程矛盾：请求有语义，网关看不见

考虑以下两条经过同一网关的请求：

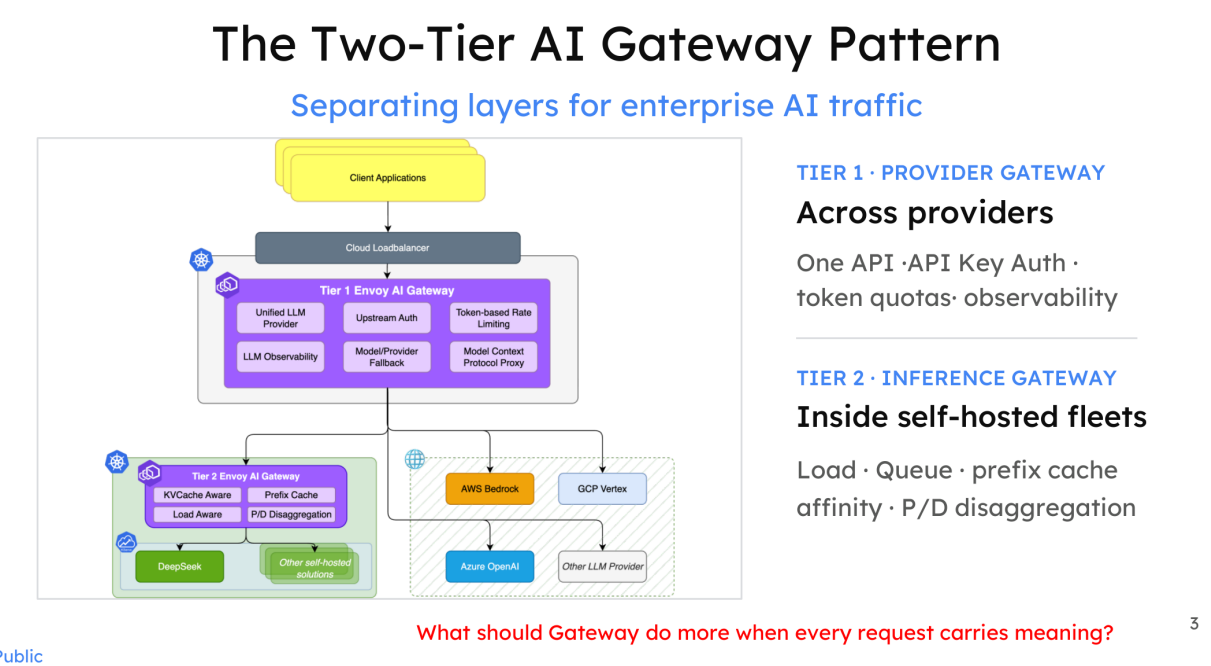
- 请求 A：“你好，今天天气怎么样？”- 简单问候，任意轻量模型即可胜任。
- 请求 B：“请为以下分布式系统设计一个基于 Raft 共识的容错方案，并给出伪代码。”- 需要深度推理能力的复杂生成任务。

两条请求的 Token 长度可能相近，鉴权与限流环节无法区分它们，负载均衡器根据队列最短或缓存亲和度将它们分配到同一高规格实例。结果：请求 A 占用了本可服务请求 B 的大模型算力，而请求 B 若被分配到小模型则可能因能力不足产出低质量结果。算力错配（Compute Mismatch）在此成为系统性问题，而非偶发事件。

要理解这一矛盾的根源，需要先看清当前 AI 基础设施已经演变出的两层网关架构，再审视模型与算力生态正在经历的多维碎片化。

两层 AI 网关：从传统代理到推理调度

下图展示了社区提出的两层 AI 网关模式，也是当前大多数企业 AI 流量管理的基线架构。



图注：两层 AI 网关模式（Two-Tier AI Gateway Pattern）的总体架构。来源：演讲 PPT 第 3 页

图中将企业 AI 流量的管理职责拆分为两个独立层级：

层级	名称	核心职责	典型关注指标
Tier 1	供应商网关（Provider Gateway）	统一 API 接入、API Key 鉴权、Token 级限流、可观测性	每分钟 Token 配额、请求计数
Tier 2	推理网关（Inference Gateway）	自建集群内负载感知、队列调度、前缀缓存亲和（Prefix Cache Affinity）、P/D 分离路由	GPU 队列深度、KV Cache 使用率

箭头显示了流量从客户端经过云负载均衡器后的两条路径：一条流向外部供应商（如 AWS Bedrock、GCP Vertex、Azure OpenAI 等），由 Tier 1 完成鉴权与限流；另一条流向自建推理集群（如部署了 DeepSeek 等模型的 GPU Fleet），此时 Tier 2 接管，利用后端 Metrics 进行更细粒度的调度。

这一架构的演进逻辑清晰：传统 API 网关本质上是反向代理技术（Nginx、Envoy 等）在新工作负载上的延伸 - 后端从普通微服务变成了 vLLM（一种高吞吐大模型推理服务引擎）、SGLang（一种推理引擎框架）等部署的模型实例，限流粒度从"每分钟请求数"细化到"Token 级速率限制"。Tier 2 的出现则是因为跨 Worker 的请求级调度策略与 Tier 1 的供应商管理逻辑截然不同，必须独立处理。

然而，两层架构的盲区恰恰在于：所有调度信号 - Token 配额、并发数、队列深度、缓存命中率 - 全部是资源侧指标，没有任何一项能反映请求本身的意图或复杂度。

四维碎片化：网关面对的组合爆炸

下图以引入混合模型架构之前的视角，展示了应用与底层资源之间已经形成的密集交叉连线。

Fragmentation Changed the Problem

The model ecosystem becomes fragmented.

MODEL

large / small / hybrid reasoning /
specialized / open / closed

COMPUTE

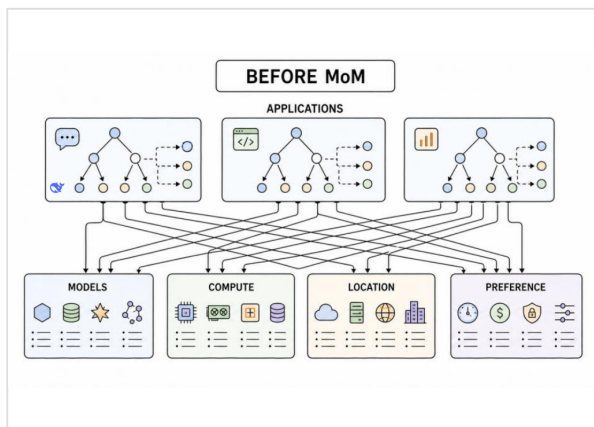
CPU / GPU / NPU · heterogeneous
cost and energy

LOCATION

cloud / data center / edge

PREFERENCE

quality / latency / cost / privacy



After DeepSeek open source its models



Gateways know providers and
replicas. It is the best fit place

4

Public

图注：碎片化改变了问题本质 - 应用与资源之间形成复杂的多对多映射。来源：演讲 PPT 第 4 页

碎片化（Fragmentation）沿以下四个维度同时展开：

- 1 模型（Model） - 大模型 / 小模型 / 混合推理 / 专用模型 / 开源 / 闭源，能力差距和架构差异持续扩大。
- 2 算力（Compute） - CPU / GPU / NPU，不同硬件在成本、能耗和模型适配度上差异显著。
- 3 位置（Location） - 云端 API、自建数据中心、边缘设备各有部署诉求。
- 4 偏好（Preference） - 质量优先、延迟优先、成本优先、隐私优先，不同业务场景的优先级完全不同。

四个维度的笛卡尔积使得"哪个请求该发往哪个模型、跑在哪块硬件上"成为一个组合爆炸问题。而两层网关的全部调度信号均无法触及请求的意图和复杂度。

小结

两层网关架构解决了供应商统一接入和集群内负载均衡的工程问题，但它的调度语言止步于流量指标。面对四维碎片化的生态，仅凭 Token 计数和队列深度无法完成"请求 - 模型 - 硬件"的最优匹配。网关必须进化出理解请求语义的能力 - 这意味着在现有 Tier 1 与 Tier 2 之间，需要插入一个专门的语义层，将请求的意图、复杂度和偏好转化为可供路由决策使用的结构化信号。

二、引入语义层：vLLM-SR 的系统定位与职责边界

新增组件是否意味着更多耦合？

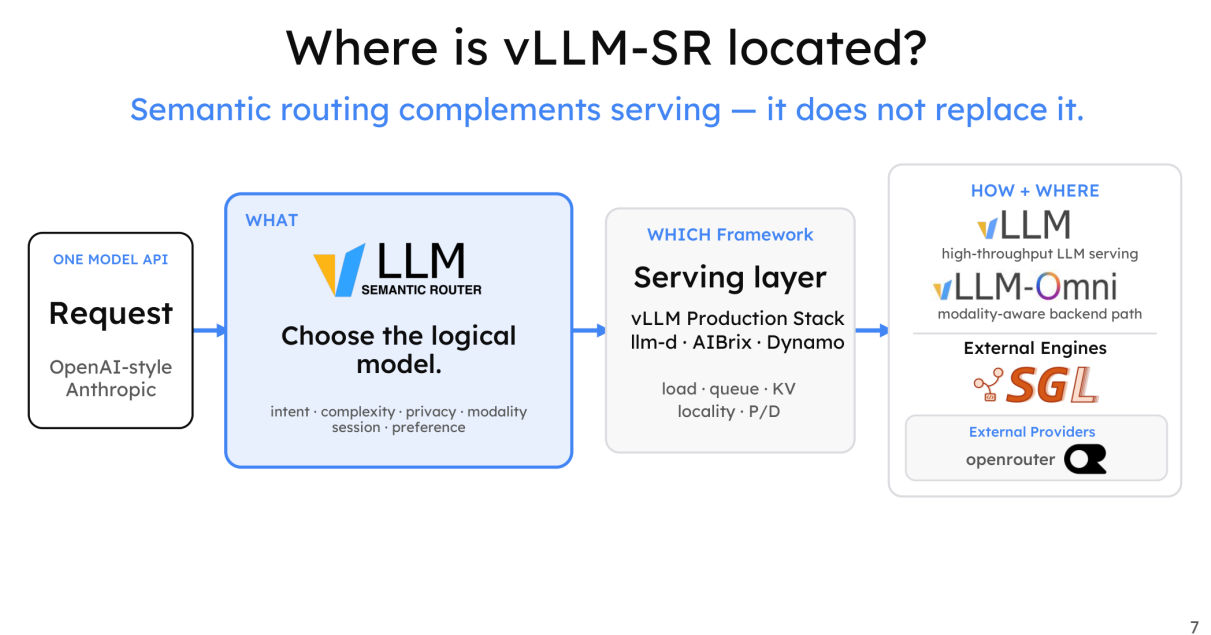
在已有推理引擎与网关之间再插入一个“语义路由器”，一个自然的工程担忧是：这是否会增加系统耦合度，甚至与推理引擎争夺调度权？

这个问题的根源在于**职责边界模糊**。传统代理或网关往往把“选哪个模型”和“怎么调度请求”混在同一层完成。一旦混合模型池规模增长，这两类决策的输入信号完全不同
前者依赖语义（意图、领域、风险），后者依赖系统状态（队列深度、KV 缓存局部性、设备利用率）。将它们强行耦合，会使任何一端的迭代都牵动另一端。

vLLM-SR 给出的回答是：把“选什么模型”独立为一个透明语义层（Semantic Layer），与下游推理引擎各自专注各自的问题。

请求生命周期中的三阶段

下图展示了一个请求从进入统一 API 到最终被执行的完整路径，明确了语义路由器在栈中的位置。



图注：vLLM-SR 系统定位示意图，展示请求生命周期的三个阶段。来源：演讲 PPT 第 7 页

图中将请求生命周期切分为三个显式阶段：

阶段	关键问题	负责组件	决策依据
WHAT（选什么）	选择逻辑模型	vLLM-SR	意图、复杂度、隐私、模态、会话偏好
WHICH（用什么框架）	确定服务层	vLLM Production Stack / llm-d / AIbrix / Dynamo	框架能力、模态后端匹配

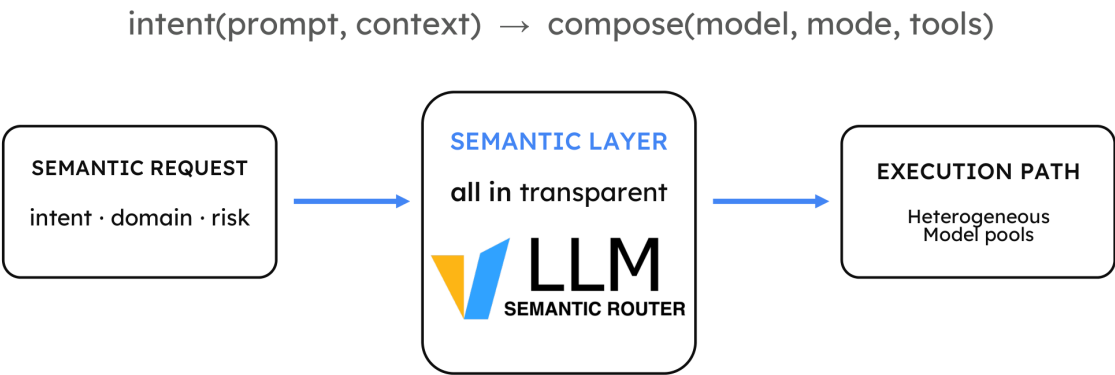
阶段	关键问题	负责组件	决策依据
HOW + WHERE（怎么执行、在哪执行）	负载均衡、KV路由、P/D 分离	vLLM / SGLang / 外部供应商	队列长度、KV局部性、设备利用率

箭头方向体现了关键因果关系：请求首先到达语义层，由 vLLM-SR 根据语义信号决定逻辑模型（WHAT），随后才被转发到具体的服务层与推理引擎。语义决策发生在执行之前，而非执行之中。

透明语义层的工作逻辑

下图进一步说明了语义层本身如何将请求意图映射为执行路径。

What if we build a router which understands the semantics?



Pre-allocate the intelligence before forwarding it.

5

Public

图注：语义层将请求意图映射为执行路径的流程。来源：演讲 PPT 第 5 页

图中左侧的语义请求（Semantic Request）携带三类信号：intent（意图）、domain（领域）、risk（风险等级）。中间的语义层将这些信号翻译为一条具体的执行路径（Execution Path），指向右侧异构模型池中的某个逻辑模型。核心流程可表达为：

intent(prompt, context) → compose(model, mode, tools)

关键词是“透明（transparent）”：语义层不改变请求格式，也不持有推理状态。它在请求被执行前完成智能预分配，然后将标准格式请求原样递交给下游。对于下游 vLLM 或 SGLang 实例而言，收到的仍然是一个普通的 OpenAI 风格或 Anthropic（AI 模型供应商）风格的请求 - 它们无需感知上游存在语义路由。

最小例子与扩展

在最简单的单卡部署场景中：用户发送一条低复杂度的事实检索查询，vLLM-SR 分析意图后将请求映射到轻量逻辑模型端点，下游实例按自身负载策略执行。两个阶段各自独立决策。如果将来推理引擎从 vLLM 替换为 SGLang，语义层无需修改 - 因为它只输出"选什么模型"的结论，不关心模型由谁执行。

当场景扩展到多卡或多节点时，新增变量是模型池的异构性（不同硬件、不同地域、不同供应商）。语义层增加的决策维度是隐私和模态，而负载均衡的复杂度增长完全由服务层吸收，两层的迭代节奏互不干扰。

需要注意的是，演讲材料未给出 vLLM-SR 引入后的端到端延迟开销数据，因此语义层的"透明"程度在量化层面尚需后续基准测试验证。

小结

语义路由是补充而非替代。vLLM-SR 通过将"选什么模型"从推理引擎中剥离，形成独立的语义层，使两侧可以沿各自的技术轨道独立演进。系统定位明确后，下一步自然要问：一个具体的路由决策究竟能带来多大收益？

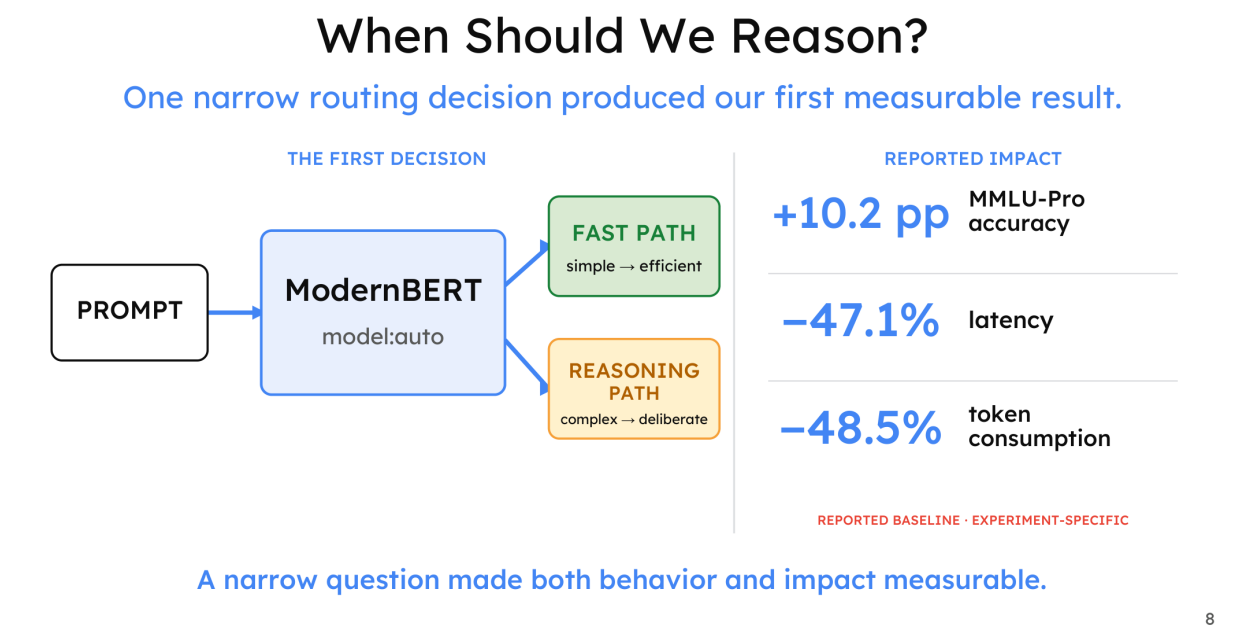
三、最小可行解：基于 ModernBERT 的双分支路由

推理能力与资源消耗的两难

具备深度推理能力的模型往往伴随高延迟和大量 Token 开销，而轻量快速的模型在复杂任务上准确率不足。现实流量中大部分请求并不需要深度推理 - 简单事实查询、格式转换、基础问答占据了相当比例。如果对所有请求统一调度重型模型，资源会被严重浪费。由此引出核心问题：能否在请求到达推理引擎之前，用一个极低开销的判别器将流量分流？

双分支架构

下图展示了这一最小可行方案的完整流程及实验指标，它是 vLLM-SR 最早的工程原型。



Public

图注：双分支路由架构与实验指标。来源：演讲 PPT 第 8 页

图中包含三个关键元素和一条核心判别逻辑：

图中元素	角色	说明
PROMPT	入口	用户提交的原始查询
ModernBERT	判别器	基于编码器架构（Encoder-based Model）的轻量分类模型，对 Prompt 做二分类，推理开销通常在毫秒级
FAST PATH	快速分支	标记为 simple → efficient，承接简单查询
REASONING PATH	推理分支	标记为 complex → deliberate，承接需深度思考的查询

箭头方向体现因果链：Prompt 首先进入 ModernBERT，由其分类结果决定下游走哪条路径。

为什么简单分流能同时改善三个指标？

双分支路由的收益来自一条清晰的因果链：

- 1 减少不必要的推理深度：简单查询被导向快速模型，避免触发长链推理过程。
- 2 延迟下降：快速模型的首 Token 响应时间和总生成时间均显著低于推理模型。
- 3 Token 消耗减少：推理模型在思考过程中会产生大量中间 Token（如思维链输出），简单查询绕过总消耗下降。
- 4 准确率反升：复杂查询不再与简单查询争抢同一模型的注意力预算，推理模型得以专注处理真正需要深度思考的问题。

本质是让路径上的模型与任务难度匹配，而非让单一模型承担所有复杂度梯度。

数据锚点：受控实验中的结果

在一项针对特定路由决策的受控实验中（以 MMLU-Pro 基准为基础），双分支路由产生了如下结果：

指标	变化幅度
MMLU-Pro 准确率	+10.2 个百分点
延迟	-47.1%
Token 消耗	-48.5%

MMLU-Pro（Massive Multitask Language Understanding - Professional，大规模多任务语言理解专业版）是一个覆盖多学科的评测基准，此处用于衡量路由前后任务准确率的变化。

必须强调的限定条件：这些数字来自范围狭窄的路由决策实验，原始材料明确标注为实验特定基线（Experiment-Specific）。演讲材料未给出对照模型名称、两条路径分别使用了哪些模型，以及测试集的规模与分布细节。因此，这些数字不宜直接外推到任意生产场景。

局限与扩展瓶颈

双分支路由虽然在上述受控条件下效果显著，但作为最小原型存在明确的扩展性瓶颈：

- 分类粒度单一：仅做“简单/复杂”二分类，无法识别查询所属领域（数学、编程等），也无法判断是否包含安全风险。据演讲者描述，早期原型实际需要训练三个独立编码器分别处理领域分类、越狱检测和 Token 级标注 - 单一二分类器很快就不够用了。
- 静态决策边界：分类阈值在训练时固定，无法根据实时负载或模型可用性动态调整。
- 缺乏反馈闭环：路由决策的质量无法被下游结果回传修正，错误分流会直接导致质量劣化且无从察觉。

小结

基于 ModernBERT 的双分支路由验证了一个核心命题 - 在推理引擎前端增加一层极低开销的语义判别，足以在特定条件下同时改善准确率、延迟和成本三个通常相互矛盾的指标。但它本质上是粒度粗糙的静态分类器。面对领域识别、安全过滤、多模态等真实需求时，路由器需要向更丰富的信号处理架构演进。

四、架构演进：突破静态瓶颈的信号驱动决策流

静态流水线为何无法扩展

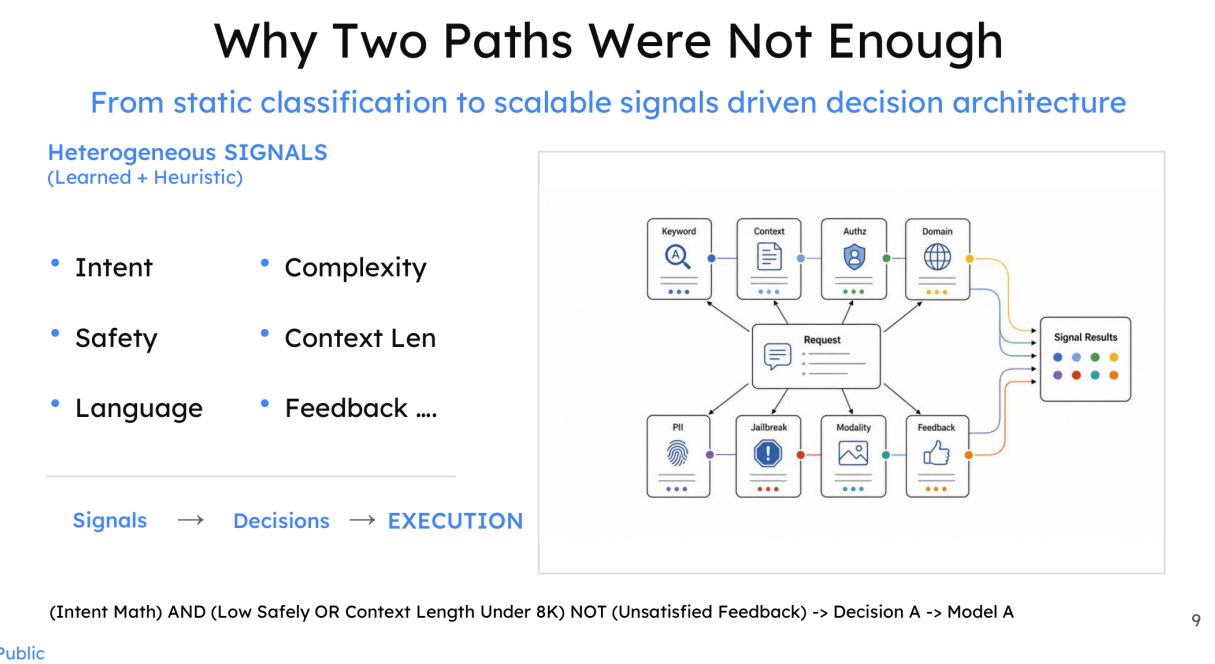
最初的语义路由器采用一条串行流水线：越狱检测（Jailbreak Classifier）→ 隐私实体识别（PII Token-level Classifier）→ 领域分类（Domain Classifier）。三个模型前后衔接，越狱命中即拦截，PII 泄露即拒绝，剩余请求由领域分类器分发。

然而，当路由器尝试接入模型数量急剧增多的动态环境时，串行管道暴露出根本性矛盾：可接纳的信号维度和可选择的模型数量都被管道的拓扑结构硬编码了。据演讲者描述，HuggingFace 推出 HuggingChat Omni 模式后需要在平台上动态选取模型，而当时的静态流水线无法承载 - 可用模型池从个位数跃升至数十甚至上百，串行三步无力覆盖。

这一矛盾推动了架构的核心转型：从静态分类流水线走向可扩展的信号驱动决策架构（Signal-driven Decision Architecture）。

异构信号的并行提取

解决扩展问题的第一步，是重新定义路由器观察一个请求的方式。下图展示了从串行管道到并行信号提取的结构性变化。



图注：请求经多个信号提取器并行处理，输出汇聚为信号结果矩阵，再经布尔逻辑生成路由决策。来源：演讲 PPT 第 9 页

图中将一个进站请求分发给多个并行的信号提取器 - 包括 Keyword、Context、Authz、Domain、PII、Jailbreak、Modality、Feedback 等 - 输出汇入统一的信号结果矩阵（Signal Results）。信号被显式分为两类：

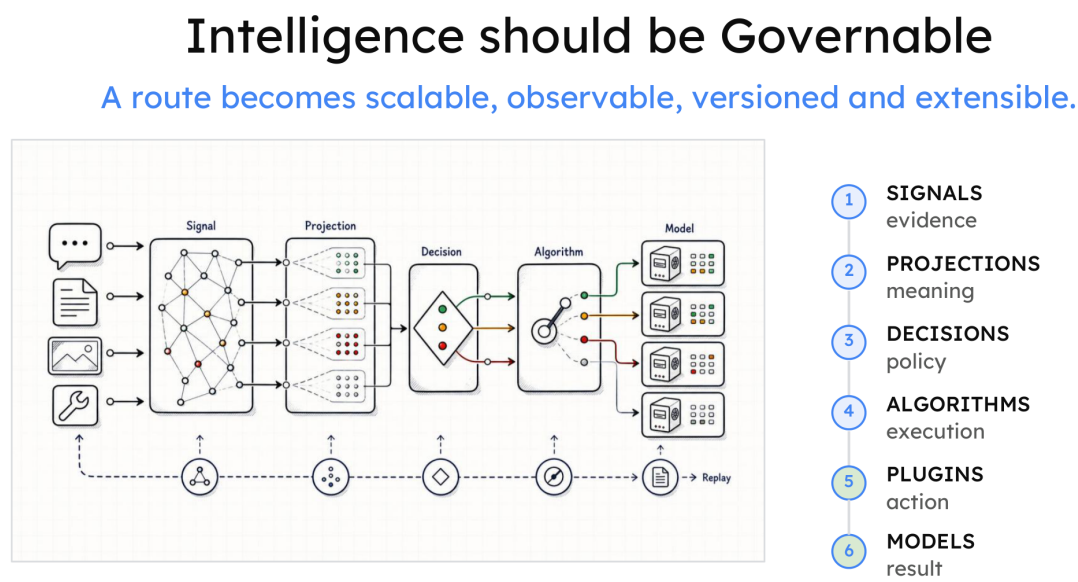
类型	特征	典型信号
启发式信号 (Heuristic)	确定性 100%，无需模型推理	关键词匹配、上下文长度、语言识别、结构化字段
学习式信号 (Learned)	概率输出，依赖分类器或编码器	意图分类、越狱检测、PII 实体标注

关键改变在于串行变并行。原先越狱 -> PII -> 领域的串行依赖被消解：一个请求可能同时携带越狱风险、PII 信息和特定领域特征，三个维度的观测互不阻塞、各自独立产出信号值。并行化带来两个直接收益：延迟降低（总延迟取决于最慢的单个提

取器，而非所有提取器之和）；维度可扩展（新增观测维度只需添加一个提取器，不影响已有管道）。

六阶段流水线：从信号到模型

信号提取只是第一步。完整的路由决策还需回答：观测结果意味着什么？该执行什么策略？用哪个模型？下图将路由过程分解为六个可独立迭代的阶段。



10

Public

图注：六阶段路由流水线，实现可观测、可版本化、可扩展的路由治理。来源：演讲 PPT 第 10 页

阶段	名称	职责关键词	输入 → 输出
1)	信号 (Signals)	证据	原始请求 → 多维信号值
2)	投影 (Projections)	含义	信号值 → 语义映射（如将 token 数映射为"长/短"）
3)	决策 (Decisions)	策略	投影结果 → 布尔组合策略
4)	算法 (Algorithms)	执行	策略 → 路由执行逻辑
5)	插件 (Plugins)	动作	执行逻辑 → 具体系统调用
6)	模型 (Models)	结果	系统调用 → 模型推理输出

阶段之间构成单向依赖链，每个阶段仅消费上游输出、生产下游输入。核心因果机制是解耦：信号层的变更（如增加 Feedback 信号）不需要修改决策层逻辑；决策层的策略调整不影响底层模型部署。

布尔组合：将异构信号转化为路由策略

投影层将原始信号值标准化为可判断的语义标签后，决策层使用布尔逻辑（Boolean Logic）将多个条件组合为路由策略。PPT 第 9 页底部给出了一个示例性表达式：

(Intent = Math) AND (Low Safety OR Context Length < 8K)
 NOT (Unsatisfied Feedback)
 → Decision A → Model A

拆解各谓词：**Intent = Math** 来自学习式意图分类器；**Low Safety** 来自越狱检测器的低风险判定；**Context Length < 8K** 是启发式信号，直接计数 Token 即可确定；**Unsatisfied Feedback** 取决于历史交互数据。

布尔表达式可任意组合 AND / OR / NOT，策略空间随信号维度增加而指数级扩展，无需改动流水线拓扑。例如，当安全信号上线后，只需在决策层增加一个 **Safety = High → 拒绝或转审核** 的谓词，信号层的提取器独立部署并行执行，不改动已有流程。

边界条件

- **信号冲突**：当学习式信号给出矛盾判定（如意图分类器判定"数学"而领域分类器判定"闲聊"）时，当前材料未给出具体的冲突消解机制。
- **延迟开销**：并行信号提取虽然理论上只取最慢一环的延迟，但若某个学习式分类器本身推理耗时较高，对端到端延迟的影响仍需关注。演讲材料未给出各提取器的实测延迟数据。
- **策略爆炸**：布尔条件的自由组合在维度较多时可能产生大量规则，如何管理规则版本与互斥关系是额外的工程治理问题。

小结

信号驱动决策架构将串行的三步分类拆解为并行的异构信号提取，再通过投影、布尔决策、算法、插件四个中间层将信号转化为可治理的路由策略。六阶段流水线的每一层职责单一、接口明确，新增观测维度或调整策略都不必重构全局管道。这种灵活的决策编排能力，使路由器不再局限于"选一个模型"，而是可以编排多个模型协同工作。

五、从单选到协同：多模型协作的微智能体算法

单体选择的天花板

前面各章讨论的路由核心动作都是"选一个最合适的模型"。但存在一个根本矛盾：路由再精准，单模型的能力上限不会因为被选中而提高。当请求本身的难度超出候选池中任何单一模型的可靠作答范围时 - 例如一段需要推理、搜索、再验证的复合任务 - 单次选择只能给出"最不差的答案"，而非"足够好的答案"。

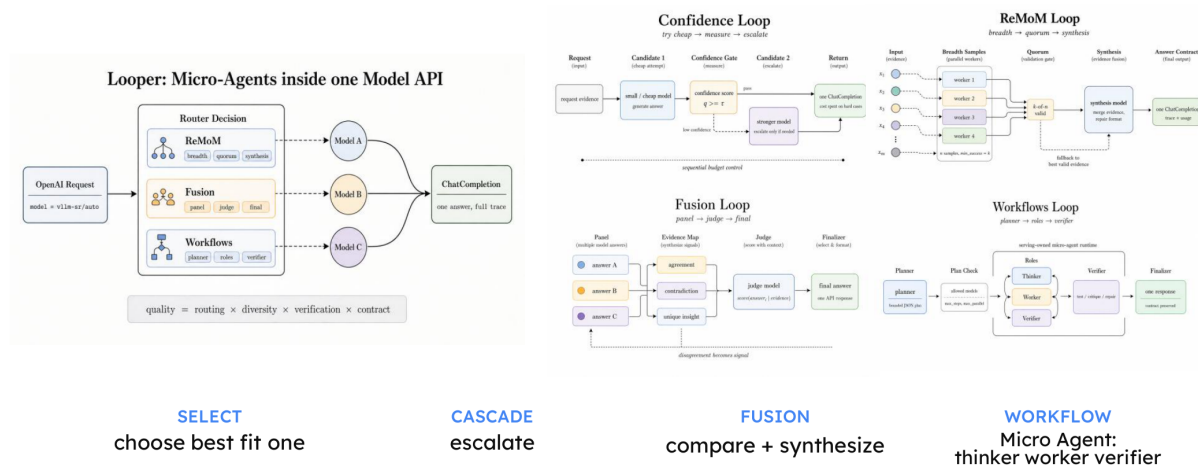
突破这一天天花板，需要路由器从 Model Selection（模型选择）演进到 Model Collaboration（模型协作）：在一次请求的生命周期内实时编排多个模型，以计算时间换取输出质量。vLLM-SR 将这类协作策略归纳为四种语义模式，并以 Looper 微智能体的形式实现。

四种协作语义全景

下图按协作复杂度从左到右排列了四种模式，每种模式对应不同的 Looper 流程和角色分工。

From Model Selection to Model Collaboration

Unlock the power of collective intelligence



11

Public

图注：多模型协作的四种语义模式及其内部 Looper 流程。来源：演讲 PPT 第 11 页

语义模式	核心动词	参与模型数	循环条件
SELECT	choose best-fit one	1	无循环，一次选择
CASCADE	escalate	≥ 2 (分级)	置信度不足时升级
FUSION	compare + synthesize	≥ 2 (并行) + Judge	Judge 判定不合格时重入
WORKFLOW	plan $\rightarrow$ execute $\rightarrow$ verify	≥ 3 (角色化)	Verifier 拒绝时重入

SELECT 已在前面章节讨论过，以下聚焦后三种协作算法。

CASCADE：置信度驱动的逐级升级

图中 Confidence Loop 的箭头流向为：请求进入后被分发给成本最低的候选模型；模型返回结果后系统做置信度评估（Confidence Check）；若达标则直接返回，若不达标则请求传递给下一级更强模型，重复评估，逐级攀升直到置信度合格或到达最强模型。

核心变量是置信度阈值与候选模型的成本梯度。阈值越低，请求越倾向于在低层级被消化 - 延迟低、成本低，但质量风险增大；阈值越高，更多请求被推升到高成本模型。演讲材料明确指出这是一种实时 (real-time)

的请求级算法，每条请求独立判定，不依赖离线批处理，但未给出具体阈值数值或线上 A/B 数据。

FUSION：多模型并行生成 + 裁判综合

Fusion 的思路与级联正交：不是串行升级，而是让多个模型同时回答同一个问题，再由一个 Judge 模型综合最终答案。

图中 Fusion Loop 的关键节点为：（1）Panel（面板） - 同一请求被广播给 N 个候选模型，各自独立生成答案；（2）Judge（裁判模型） - 收集全部候选答案，对比优劣后合成综合回答；（3）质量检查 - 若 Judge 输出 qualified 则返回，否则重入循环。

融合策略还衍生出一种 ReMoM Loop 变体：第一层用较多模型各自推理，第二层用较少模型对上一轮答案做二次推理，层层收窄最终汇聚为一个答案 - 类似锦标赛淘汰赛的拓扑，每一轮都让模型间的多样性转化为更高质量的共识。

Fusion 的新增变量是并行模型数量和 Judge 模型的选择。并行数越多答案多样性越高，但推理成本线性增长；Judge 本身若能力不足，综合质量反而可能不如最优单模型。

WORKFLOW：角色化微智能体

Workflow 是协作复杂度最高的模式，将一次请求拆分为多个角色化子任务：

- Thinker / Planner（思考者 / 规划者）：接收原始请求，理解意图并拆解为若干子问题。
- Worker（执行者）：领取子问题，执行推理并返回局部结果。
- Verifier（验证者）：汇总所有 Worker 输出做正确性验证。通过则合成最终答案返回；失败则回退给 Thinker 重新规划或给 Worker 重新执行。

这一结构与通用 Agent API 协作框架在形态上相似，但关键区别在于它运行在请求级的实时路径上，每一次调用都可能在毫秒到秒级的时间窗口内触发完整的 Plan → Execute → Verify 循环。

最小例子：一次代码生成请求的三种协作路径

假设用户提交了一条"编写一个并发安全的 LRU 缓存"代码生成请求：

- CASCADE 路径：路由器先将请求发给轻量代码模型，系统评估结果置信度偏低（如缺少锁机制），自动升级到更强的代码模型重新生成，直到置信度达标。
- FUSION 路径：请求同时发给三个不同代码模型，各自生成实现方案。Judge 模型对比三份代码，提取各自最优片段（如模型 A 的锁策略 + 模型 B 的淘汰算法），综合为终稿。
- WORKFLOW 路径：Thinker 模型将任务拆解为"数据结构设计""并发控制""单元测试"三个子任务，分别分配给擅长不同领域的 Worker 模型。Verifier 发现并发控制子模块存在死锁风险，将该子任务回退给对应 Worker 修复，最终通过验证后合成完整代码。

三条路径的成本-延迟-质量权衡各不相同：CASCADE 最节约（大部分请求在低层消化），FUSION 并行开销最大但答案多样性最高，WORKFLOW 延迟最长但对复杂任务的结构化拆解能力最强。

小结

路由器的职能已经从"给请求找一个最佳模型"的静态分发，演进为"在一次请求内动态编排多模型协作"的实时算法层。这一演进的本质是用 **Test-time Compute**（测试时计算，指在推理阶段投入额外计算资源以提升输出质量）换取超越任何单体模型能力上限的输出质量。但协作策略的每一次循环都意味着额外的推理开销 - 演讲材料未给出各策略在生产负载下的具体延迟增幅或质量增益数据，效果需要结合实际场景验证。

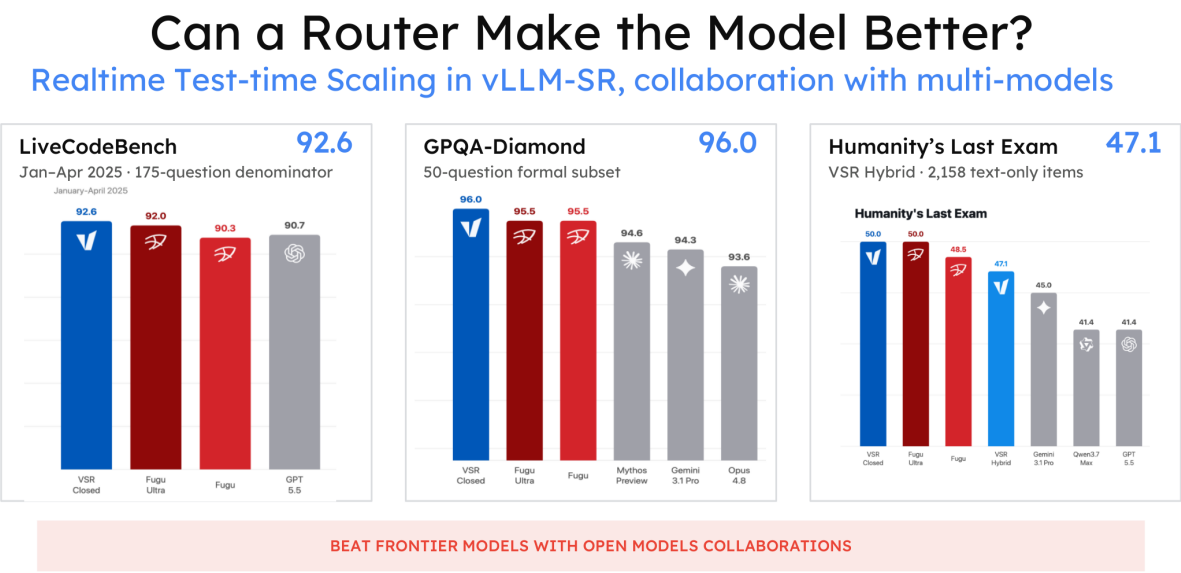
六、工业界验证：开源协作超越前沿单体的工程证据

性能天花板与成本地板能否同时突破？

企业部署大语言模型时始终面临一组对立约束 - 追求最高准确率意味着调用最昂贵的闭源前沿模型，而控制成本往往以牺牲质量为代价。多模型协作架构试图打破这一零和博弈：用路由器把不同难度的请求分配给不同成本的模型，使总体得分接近甚至超过单一前沿模型，同时将大部分流量引向低成本节点。以下从公开基准测试和工业部署两个维度检验这一假设。

基准测试：特定子集上的得分对比

下图展示了多模型协作方案与闭源前沿模型在三项基准上的得分对比，需要关注的是每项基准的子集规模。



Public

图注：vLLM-SR 多模型协作在三项基准上的得分。来源：演讲 PPT 第 12 页

图中横轴比较了 **VSR (Closed / Hybrid 两种配置)**、**Sakana Fugu**（一种多模型智能组合系统，通过进化搜索将多个开源模型的输出进行融合）以及 **GPT-5.5**、**Gemini 3.1 Pro**、**Opus 4.8** 等闭源前沿模型。核心数据：

基准	子集规模	VSR Closed 得分	对比观察
LiveCodeBench	175 题（2025 年 1 - 4 月子集）	92.6	图中高于 GPT-5.5 对应柱状条
GPQA-Diamond	50 题（正式子集）	96.0	图中高于多个前沿模型柱状条
Humanity's Last Exam	2,158 条纯文本	47.1（VSR Hybrid）	图中未超过最高前沿模型

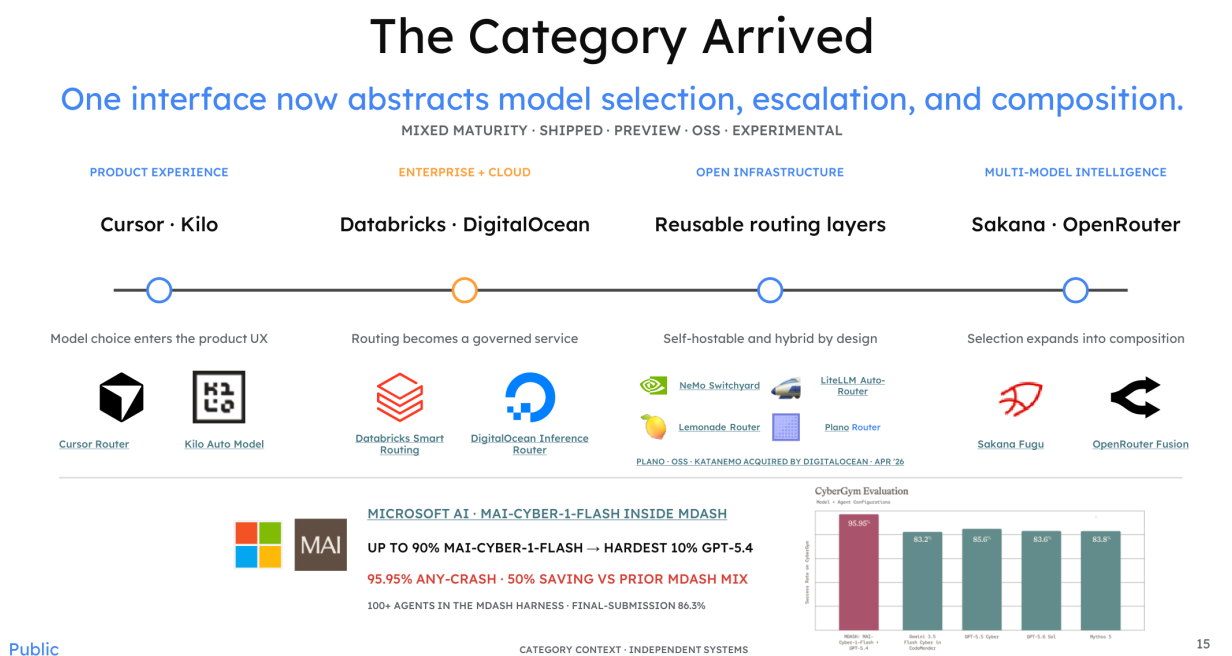
LiveCodeBench
是一个用于评估代码生成与推理能力的动态基准；GPQA-Diamond（Graduate-Level Google-Proof QA）是一个研究生级科学问答评测集。

因果解读：VSR Closed 在 LiveCodeBench 和 GPQA-Diamond 上的得分均高于图中所列多个前沿模型，这是"实时测试时缩放"（Realtime Test-time Scaling）的体现 - 路由器在测试时对每道题实时选择最适合的开源模型作答，不同开源模型各有擅长子域，路由器的选择收益得以显现。

边界条件：前两项基准的子集规模较小（175 题、50 题），统计置信区间相对宽。在样本更大的 Humanity's Last Exam（2,158 条）上，VSR Hybrid 的 47.1 并未超过最高前沿模型。换言之，协作架构的优势并非在所有基准上均成立，题目数量和领域覆盖度直接影响结论的泛化性。

工业部署：微软 MDASH 的混合路由实践

下图底部展示了 MDASH 在网络安全场景中的部署数据，将基准测试的结论推进到生产环境。



MDASH（微软 AI 模型路由与智能体框架，用于在超过 100 个智能体之间协调调度模型）在网络安全场景中采用了一条简明的混合路由策略：

高达 90% 的请求 → MAI-CYBER-1-FLASH（微软自研轻量网络安全模型）

最困难的 10% 请求 → GPT-5.4

这一分流产生的关键结果：

指标	数值
任务成功率（ANY-CRASH）	95.95%
成本节省	约 50%（对比先前 MDASH 中的模型组合）
最终提交得分	86.3%（在包含 100+ 智能体的评估框架内）

因果链：MAI-CYBER-1-FLASH 承担绝大多数常规安全分析请求，推理成本远低于 GPT-5.4。路由器仅在置信度低或任务复杂度高时将请求升级到 GPT-5.4。安全领域约 90% 的查询属于已知模式或中等难度，轻量模型即可正确处理，仅有长尾困难案例需要前沿模型介入。这与前文级联策略的核心逻辑完全一致，只是从学术基准延伸到了生产级智能体集群。

限定条件：演讲材料未给出 MAI-CYBER-1-FLASH 的单独准确率，也未公开路由判定的具体阈值。50% 的成本节省是相对于先前 MDASH 模型组合而言，非相对于 GPT-5.4 全量调用。

从基准到生产：变量对比

维度	基准测试场景	微软 MDASH 场景	变化的关键变量
模型池	多个开源模型	自研模型 + GPT-5.4	闭源模型进入组合
路由粒度	逐题	逐请求 / 逐智能体	智能体数量 > 100
评估指标	准确率	成功率 + 成本	成本成为一阶目标
子集规模	50 - 2,158 题	生产流量（未公开量级）	统计稳定性大幅提升

核心收益模式不变：将多数流量路由至高性价比节点，仅在必要时调用昂贵模型。

小结

在限定的基准子集上，开源多模型协作已能在代码与科学推理任务中超越部分前沿闭源模型的得分；在微软的工业实践中，混合路由以 90/10 的流量分割实现了 95.95% 的任务成功率和约 50% 的成本缩减。两组证据从不同角度表明协作架构的性价比优势在真实条件下可被兑现，但泛化性仍受评测子集规模、领域特征和路由算法质量的约束。

七、终极形态：混合模型 (MoM) 的虚拟化抽象

复杂性正在向客户端泄漏

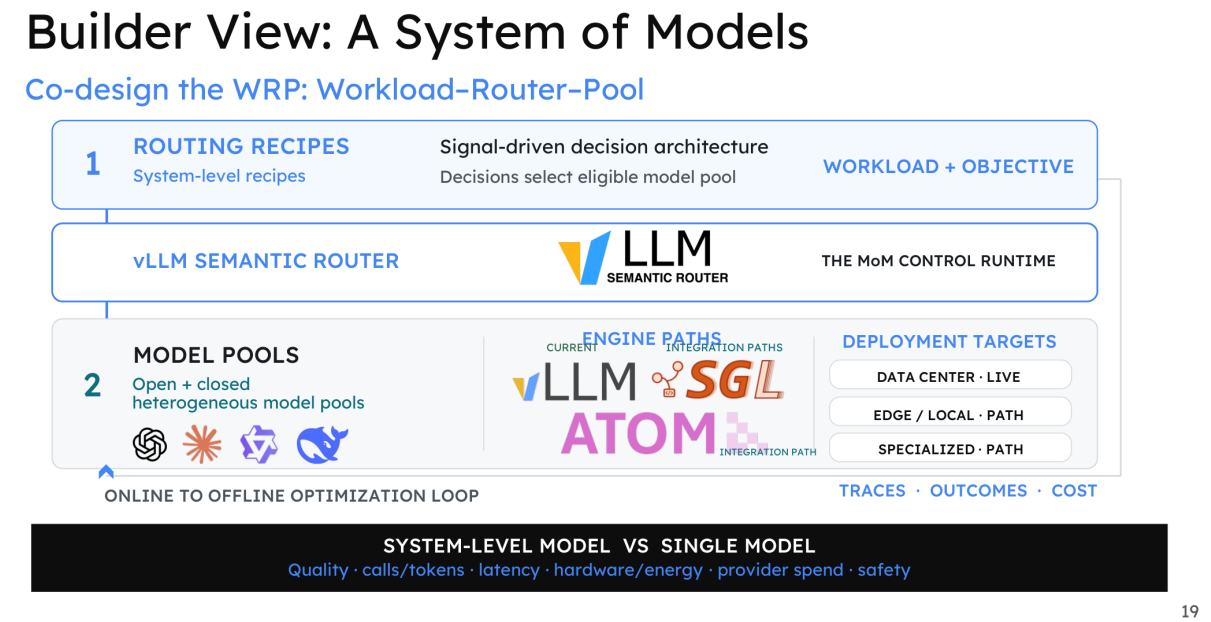
当语义路由器在后端成功地将请求分流到不同模型后，一个新的工程矛盾浮现 - 客户端该看到什么？如果每个调用方都需要知晓后端有哪些模型池、各池部署在何种硬件上、路由策略版本是多少，那么系统内部的优化收益就会被前端的集成成本所吞噬。更现实的情形是：下游调用方往往是自动化 Agent，而非人类开发者；它们只应对接一个稳定的模型端点，不应承担任何路由逻辑。

混合模型 (Mixture-of-Models, MoM，异构模型与算力池之上的单一虚拟模型契约) 正是为解决这一矛盾而提出的虚拟化抽象：对外仅暴露一个版本化的模型 ID (如 `vllm-sr/mom-balanced-v1`)，内部则由工作负载、路由算法和模型池三者协同完成全局优化。

MoM 与混合专家模型 (Mixture-of-Experts, MoE) 解决的是不同层次的问题：MoE 在单模型内部做子网络选择；MoM 在系统层面做跨模型、跨硬件的动态调度。

WRP 协同设计

下图展示了构建者视角下 MoM 控制运行时的三层架构，以及离线-在线优化闭环的运作方式。



Public

19

图注：构建者视角下的"模型系统", 展示 WRP 协同设计的三层结构。来源：演讲 PPT 第 19 页

图中自上而下包含三个核心层次，对应 WRP (Workload - Router - Pool，工作负载 - 路由器 - 模型池) 协同设计的三个组成部分：

层次	名称	职责
1)	路由配方 (Routing Recipes)	将工作负载特征与优化目标编码为信号驱动的决策架构，决定请求进入哪个候选模型池

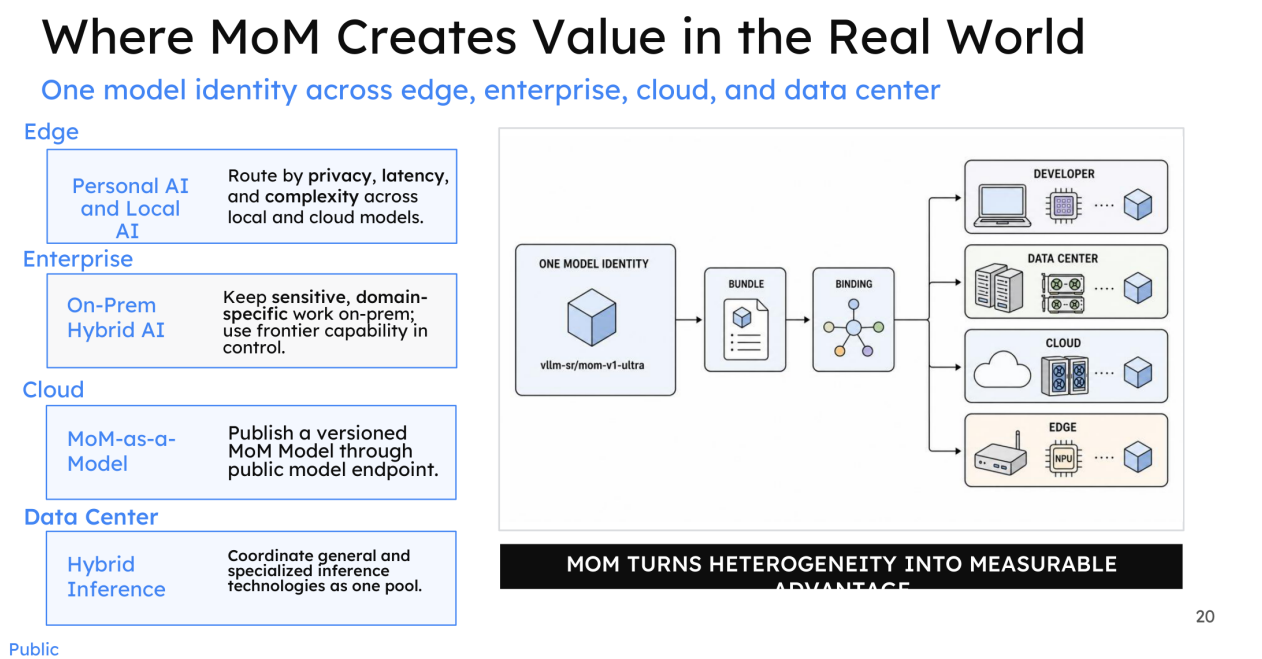
层次	名称	职责
-	vLLM 语义路由器（MoM 控制运行时）	承接路由配方输出，执行选择、级联或协作策略
2)	模型池（Model Pools）	包含开源与闭源的异构模型，通过不同推理引擎路径连接至数据中心、边缘或专用部署目标

图底部的箭头构成一条离线-在线优化闭环：运行时产生的 Traces（调用链）、Outcomes（结果质量）和 Cost（开销）数据回流至系统，驱动路由配方与模型池配置的迭代更新。评估维度在图中被显式列出 - 质量、调用/Token 数、延迟、硬件能耗、服务商支出及安全性。

因果机制：正因为 WRP 三部分被打包到同一个运行时中，客户端才能像调用单模型 API 一样调用整个系统 - 路由逻辑对外不可见，决策结果对外不可感知，仅质量与成本的最终表现可被观测。

单一模型 ID 绑定到异构环境

下图展示了同一虚拟模型 ID 如何映射到边缘、企业、云和数据中心四种场景。



部署场景	路由决策依据	典型收益
企业内网（Enterprise）	领域敏感度、合规需求	特定领域任务留在本地，必要时受控调用前沿模型
云端（Cloud）	版本化发布、弹性扩缩	以 MoM-as-a-Model 形态通过公共端点对外发布
数据中心（Data Center）	通用与专用推理技术的统一调度	将多种推理引擎的算力聚合为单一资源池

客户端始终使用同一个模型端点，后端根据请求的隐私标签、延迟预算和复杂度特征自动路由到最合适的环境和模型。

生态背景：LiteLLM（一种开源基础设施自动路由器）已在社区中提供了类似的多模型代理能力，可视为 MoM 概念在工程生态中的早期实践节点。

最小例子：Agent 视角的调用流程

以一个自动化 Agent 为例，集成过程仅需三步：

- 1 注册端点：将 `vllm-sr/mom-balanced-v1` 作为一个 model provider 接入 Agent 框架。
- 2 发送请求：按标准模型 API 格式提交 prompt，无需附加路由指令。
- 3 接收响应：返回结果与调用单一模型完全一致，Agent 无法感知后端经历了哪些路由决策。

虚拟模型的信任建立依赖两种评估视角：黑盒视角（用户/Agent 在标准 benchmark 上对比虚拟模型与基线单模型的得分，量化延迟和成本收益）和白盒视角（系统运维进入运行时内部，观测路由命中率、各池负载分布、安全拦截率等系统级指标）。演讲材料指出两种视角需同时建立，但未给出具体评估数据。

边界与适用条件

MoM 的收益与底层异构程度正相关 - 模型种类越多、硬件类型越杂、部署环境越分散，统一抽象带来的集成成本缩减与全局优化空间就越大。反之，当部署场景单一（如仅有云端一个模型）时，虚拟化抽象层的价值趋近于零，反而增加了一层间接调用开销。

小结

MoM 将全文的因果链条闭合：最初的矛盾是模型能力碎片化导致调用方选择困难；语义路由器通过意图理解实现了请求与模型的智能匹配；而 MoM 将这一匹配过程彻底封装，以单一虚拟模型契约向外界承诺稳定的质量-成本-安全权衡。从碎片化到虚拟化，系统完成了从"人选模型"到"系统替人选模型且人无感知"的架构跃迁。

结论

- 1 AI 基础设施正从单体模型服务向多模型协作范式转移。
模型、算力、位置和偏好的四维碎片化使得任何单一端点都无法最优服务所有请求。
- 2 传统两层网关的语义盲区是系统性瓶颈。Tier 1 和 Tier 2 仅处理资源侧指标，对请求意图和复杂度无感知，导致算力错配。

- 3 vLLM-SR 通过引入透明的语义层弥补了这一盲区。它专注于"选什么模型" (WHAT)，不干涉推理引擎的执行调度 (HOW + WHERE)，是补充而非替代。
- 4 信号驱动的六阶段架构使路由决策从静态分类走向动态编排。并行的异构信号提取与布尔组合策略支持维度热插拔，为多模型协作奠定结构基础。
- 5 级联、融合、工作流三种协作策略将路由器的输出从"单选"扩展为"协同"。它们以 Test-time Compute 换取超越单体模型能力上限的输出质量。
- 6 工业界数据表明，合理的模型组合在特定场景下能以更低成本达到或超越前沿闭源模型。微软 MDASH 以 90/10 流量分割实现 95.95% 成功率和约 50% 成本缩减。
- 7 MoM 虚拟化抽象最终向用户屏蔽了底层异构算力的复杂性。单一模型 ID 绑定到端、云、企多环境，客户端无需承担路由逻辑。

局限与开放问题：

- 本文涉及的基准测试数据 (LiveCodeBench 175 题、GPQA-Diamond 50 题) 来自特定子集，统计置信区间较宽，不宜直接泛化。
- vLLM-SR 语义层引入的端到端延迟开销、各信号提取器的实测耗时、协作策略在生产负载下的延迟增幅等量化数据在当前材料中均未公开。
- 信号冲突的消解机制、布尔策略的版本治理、虚拟模型的信任评估框架等工程细节有待后续实现和验证。
- MoM 的收益与底层异构程度正相关；在模型池单一的简单场景下，虚拟化抽象可能不具备足够的投入产出比。